

General Remarks:

- Make sure the code compiles and runs: `cmake` (gcc or msvc)
- Hand-in a PDF for the derivations and the plots together with the code in a zip file (no binaries).
- Respect the academic code of honor. Teams have to work on their own. Detected plagiarism results in 0 points.
- Please contact the instructor for help and utilize the Olat forum for questions.

Particle System (11 points)

Simulation Framework

In this assignment, a particle system will be simulated. A scene setup is provided as an interactive test-bed. A C++ project framework is provided that initializes simulation settings, executes the simulation loop, uses encapsulated legacy OpenGL for visualization, and `imgui`¹ for an immediate GUI. Lambda functions are provided by a viewer class for initialization, update, drawing, and the GUI: `onInit()`, `onUpdate()`, `onDraw()`, and `onGUI()`. The `glm`² library is utilized for vector computations, implementing basic operations such as addition, subtraction, multiplication, cross and dot product, etc. Any further functionality should be added to the file `exercise.cpp`. It should not be required to change any other files; also, no additional external libraries shall be included. Note that the locations of code to be inserted are marked as comments. Helper functions can be added as well.

The provided scene consists of one particle emitter that controls the initial velocity, the direction, a creation rate, and a random deviation angle. The coordinate origin is in the center of the scene and the emitter is located at $(-4.0, -1.5)^T$. Each particle stores internal variables to describe its state, see `particlesystem.h`.

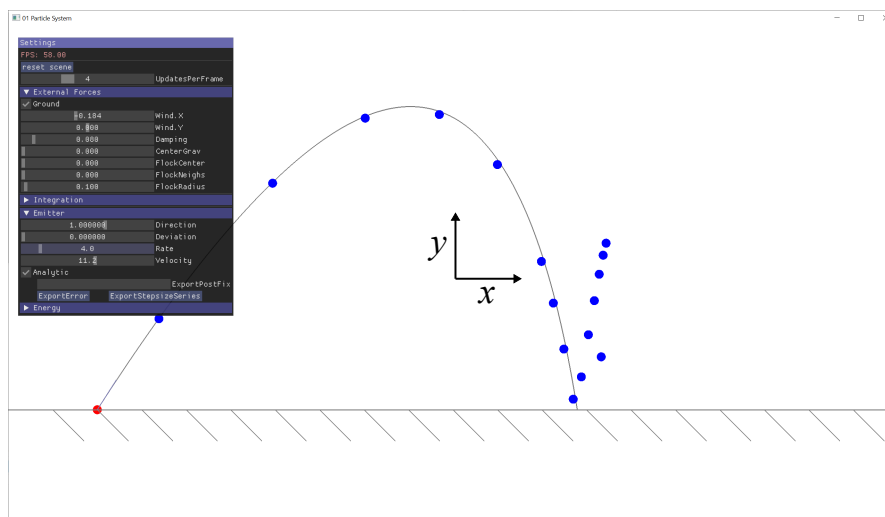


Figure 1: Scene showing simulated particles, the analytic solution, and GUI controls.

¹<https://github.com/ocornut/imgui>

²<https://github.com/g-truc/glm>

The update of the particle positions is executed based on position, velocity, and force per particle. Particles also provide a mass. Acceleration and force are related by Newton's law: $f = m a$. Different numerical solvers shall be implemented. To evaluate the quality, an analytical solution is derived for slow motions in the air. The methods and different parameters are compared to the analytic solution including linear damping and a wind force. More forces and a neighborhood based particle behavior (flocking) is finally enabled.

Compile and Run

Use a cmake toolchain for compilation; gcc and msvc were tested. On the RR26 computers more the wsl subsystem in windows has to be used:

Double click wsl shortcut on Desktop

```
sudo yum install libXrandr-devel.x86_64 libXinerama-devel.x86_64
libXcursor-devel.x86_64 libXi-devel.x86_64 mesa-libGL-devel.x86_64
```

```
unzip OptiNumAssignN_1.zip
cd OptiNumAssignN_1
cmake -Bbuild .
(-DCMAKE_BUILD_TYPE=Release)          (optionally compile optimized)
cd build
make
./bin/particlesystem
```

Tasks

The exercise consists of implementing the missing update rules of the simulation step using different settings and methods. Any additions to the code should be limited to the file `exercise.cpp`. Make yourself familiar with the provided files (`particlesystem.h/cpp`) in before. Note that in the equations vectors are set in **bold**.

- (1) (2 Points) Derive the equation of motion to simulate the trajectory of one particle. To add more realism include linear damping dependent on the velocity into the acceleration term. Show analytically how the 1D solution below is derived from the ansatz: $a(t) = A - \kappa v(t)$. Express the integration constants by the initial velocity v_0 and the initial position x_0 . Include gravity and an external force (e.g. wind) in the A of $a(t)$.

$$x(t) = \frac{1}{\kappa} \left(At + \left(v_0 - \frac{A}{\kappa} \right) (1 - e^{-\kappa t}) \right) + x_0$$

with x_0 the starting position, v_0 the starting velocity, $\kappa = \frac{k}{m}$, m mass, and k a damping constant.

Hints: Separate the equation by variables before executing the first integration. Note that $e^{a+\ln b} = b \cdot e^a$; with a, b being constants.

- (2) (0.5 Points) Provide the undamped and the damped analytic solution by implementing the two functions `posUnDamped()` and `posDampedStokes()`. Note that the 1D solutions can be treated separately for x and y . The result will be drawn as grey line if the checkbox **Analytic** is marked in the GUI interface.
- (3) (1 Point) The particle positions are controlled by forces acting on each particle. Complete the function `updateExtForces()` to support four effects: gravity downwards via the gravitational acceleration $g = 9.81m/s^2$ (negative y-direction), wind-force (x and y), a collision force at the ground level of $y = -1.5$, and a gravitational center at $x_c = (0, 0)^T$.

Employ a stiffness of $k_{\text{pen}} = 10000$ to compute the penalty force for the ground collision. To implement the gravitational center, employ Newton's law of universal gravity:

$$\mathbf{f} = \frac{G m_c m_i}{\|\mathbf{x}_c - \mathbf{x}_i\|^2} \text{nrm}(\mathbf{x}_c - \mathbf{x}_i)$$

with $\mathbf{x}_i$ the position of a particle, m_* center and particle masses, `nrm` an operator to yield a normalized (unit) vector, and G the gravitational constant. Employ the user parameter `sim.extForces.centerGravity` for the term $G \cdot m_c$ in the formula. Sum all force components up for each particle and update the particle's `force` value. Skip the update for particles that are fixed (where `points[i].fixed == true`).

- (4) (1 Points) Implement the explicit and symplectic Euler method within `computeTimeStep()` to update the velocities and positions of the particles based on their force. You may introduce helper functions such as `updatePositionsEuler()` and `updateVelocitiesEuler()` to improve readability and code-reuse.
- (5) (0.5 Points) Create two plots using a time step size of 0.05 and 0.001 showing the errors of the explicit and symplectic Euler methods towards the analytic solution including damping and wind. Use the provided functionality via the Button `ExportError` to write out a `.csv` file with the columns: `'time, x, y, aX, aY, DX, DY, Error'` (time, xy position particle, corresponding xy position of the analytic solution, and the error as distance between the positions). Compile the errors (ordinate) over time (abscissa) as line plots; one line per method.
- (6) (0.5 Point) Compute the total energy of the particle system as the sum of potential and kinetic energies per particle i :

$$E_{\text{kin},i} = \frac{1}{2} m_i (\mathbf{v}_i \cdot \mathbf{v}_i) \quad E_{\text{pot},i} = g m_i h_i,$$

with m_i mass, $\mathbf{v}$ velocity, and h height over ground. Implement the provided functions `computeKineticEnergy()` and `computePotentialEnergy()`. What can be observed in the energy plot (GUI) regarding integration methods, forces, and damping?

- (7) (2 Points) Extend `computeTimeStep()` for integration by the Verlet and Runge Kutta 4 (RK4) methods. Information can be cached by local `static` variables for the next computational step. Add both methods in the error/time plots above.
- (8) (1.5 Points) Implement a function that computes the final error at the impact position of one particle. Read through the `exportErrorOverTime()` function. Create a new function `errorAtImpact()`, which takes (`ParticleSystem& sim, float dt, ParticleSystem::eMethod method`) and returns the error and a required computation time of the integration dependent on the time step size `dt` and the integration method. Employ `glfwGetTime()` to retrieve an actual time stamp in milliseconds and use the provided array `sim.AllMethods` to iterate over the integration methods.
In `viewer.onGui()`, add the code for the button-press of `ExportStepsizeSeries` exporting a `.csv` file exporting the error measurements by columns for time step size (`dt`), final error (per integration method), computation time (per integration method).
Create two line plots: one showing the errors per method and one showing the computation time per method over the time step size. Vary the time step size from 0.0001 to 0.05. What method at what time step size would be a good choice for the simulation?
- (9) (2 Points) Further extend `updateExtForces()` by forces to simulate particle flocking. Flocking behaviour leads to particles forming groups or swarms to simulate motions such as of flying birds. Flocks of birds tend to synchronize their velocities and stay close to each other. The algorithm³ computes accelerations per particle dependent on a local

³ Reynolds (1987), *Flocks, Herds, and Schools: A Distributed Behavioral Model* from House & Keyser (2017), *Foundations of Physically Based Modelling & Animation*

neighborhood, i.e. distance and angular distributions. Three accelerations are computed to simulate collision avoidance, velocity matching, and centering:

$$\mathbf{a}_{ij}^a = -\frac{k_a}{d_{ij}} \text{nrm}(\mathbf{x}_{ij}) \quad \mathbf{a}_{ij}^v = k_v (\mathbf{v}_j - \mathbf{v}_i) \quad \mathbf{a}_{ij}^c = k_c \mathbf{x}_{ij}$$

with a_{ij} the partial acceleration on particle i due to neighbor particle j , $d_{ij} = \|\mathbf{x}_{ij}\|$ the distance between the particles, nrm an operator to yield a normalized (unit) vector, and k_* user control parameters: `eF.flockAvoidance` (k_a), `eF.flockMatching` (k_v), `eF.flockCentering` (k_c). The figure below illustrates one point and one of its neighbor with the related labeled vectors.

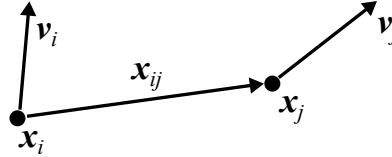


Figure 2: Reference position of a particle i and one of its neighbors j .

Each acceleration is weighted dependent on distance and angle to their neighbors. Therefore, two distances (radii) and two angles are used as parameters for steering within the weighting functions ω_d and ω_θ :

$$\omega_d(d) = \begin{cases} 1.0 & \text{if } d < r_1, \\ \frac{r_2 - d}{r_2 - r_1} & \text{if } r_1 \leq d \leq r_2, \\ 0.0 & \text{if } d > r_2 \end{cases} \quad \omega_\theta(\theta) = \begin{cases} 1.0 & \text{if } -\theta_1/2 < |\theta| < \theta_1/2, \\ \frac{\theta_2/2 - |\theta|}{\theta_2/2 - \theta_1/2} & \text{if } \omega_1/2 \leq |\theta| \leq \omega_2/2, \\ 0.0 & \text{if } |\theta| > \theta_2/2 \end{cases}$$

The per point accelerations then are defined as follows:

$$\begin{aligned} \mathbf{a}_i^a &= \sum \omega_\theta(\theta_{ij}) \omega_d(d_{ij}) \mathbf{a}_{ij}^a \\ \mathbf{a}_i^v &= \sum \omega_\theta(\theta_{ij}) \omega_d(d_{ij}) \mathbf{a}_{ij}^v \\ \mathbf{a}_i^c &= \sum \omega_\theta(\theta_{ij}) \omega_d(d_{ij}) \mathbf{a}_{ij}^c \end{aligned}$$

A final acceleration per point can be obtained by summing the three components.

Note: To execute distance queries an *octree* data structure is provided and updated in each simulation step. It results a `std::multimap` providing squared distances (`double`) and point indices (`size_t`); use as follows:

```
void updateExtForces( ... & points, ... , ... & tree ) {
    // ...
    for (size_t i = 0; i < points.size(); i++) {
        multimap<double, size_t> res; //<- prepare result container
        const vec3& p = points[i].position;
        tree->getEuclideanRangeFine( p, radius2, res ); //<- query

        for (const auto& r : res) { //<- iterate close neighbors
            // .. skip fixed points and self ...
            xij = points[r.second].position - p.position;
            dij = sqrt( r.first );
            // ...
        }
    }
}
```

Use the provided functions of the `glm` library for vector calculus, e.g. `length()`, `normalize()`, `unit()` and `dot()`. Also note that the cosine of the angle between two vectors **a** and **b** is computed, e.g., by $\cos \varphi = \text{nrm}(\mathbf{a}) \cdot \text{nrm}(\mathbf{b})$.

- (10) (1 Bonus Point) Show a comparison of computation time of employing a brute force neighborhood search (loop over all particles) and using the *octree* data structure dependent on the number of particles (plot).